Peripheral Interfacing devices

LED

7-segment display

Real world interface: ADC, DAC, SENSORS

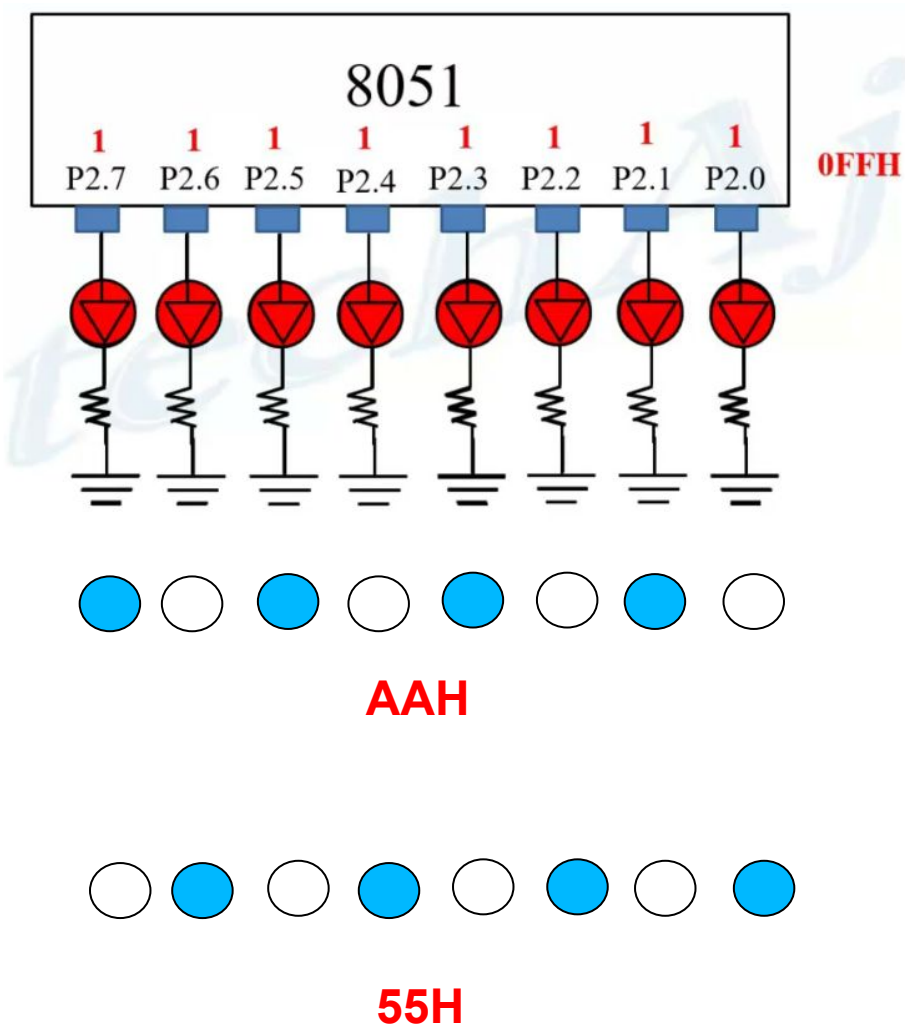
Keyboard

LCD

LED interface

Let, alternate LEDS ON/OFF

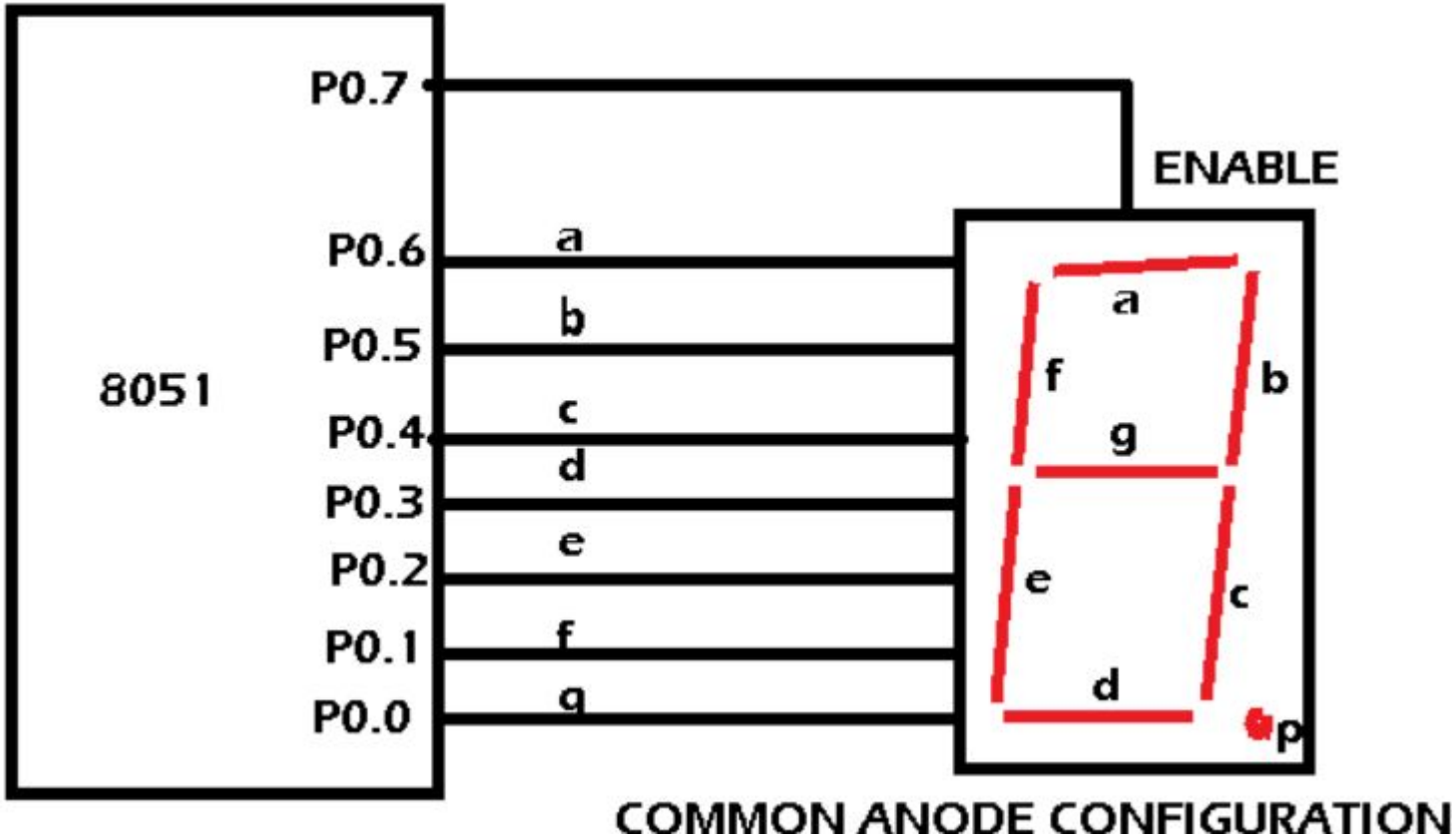
Send: AAH [10101010]
and 55H [01010101] with some delay




```
        ORG 0000H
L1:     MOV P2, #AAH
        LCALL D1
        MOV P2, #55H
        LCALL D1
        SJMP L1

D1:     DELAY PROGRAM
        WRITE HERE
```

7-segment LED interface



Common Anode: 0 □ glow; 1 □ not glow

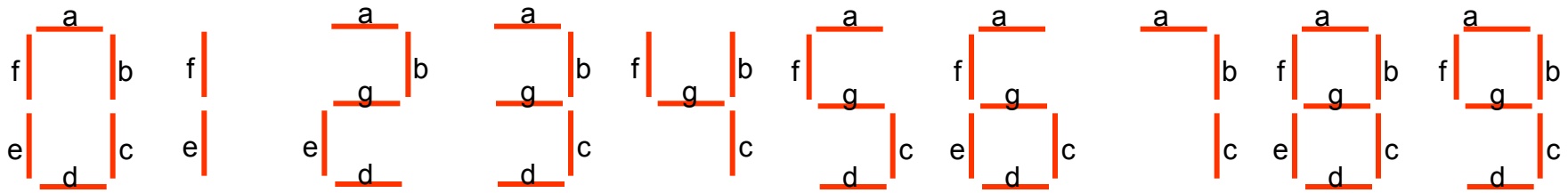
Common Cathode: 1□ glow; 0 □ not glow

BCD to 7_Seg lookup table

```
            mov     a,p3
      anl     a,0fh
get_code: mov DPTR, #7s_tab
      movc    A, @A+DPRT
      mov     p1,a

7s_tab:   db  3fh,30h,5bh,4fh,66h
      db  6dh,7dh,07h,7fh,6fh
      END
```

BCD	p g f e d c b a 7_seg	hex
0000	0 0 1 1 1 1 1 1	3f
0001	0 0 1 1 0 0 0 0	30
0010	0 1 0 1 1 0 1 1	5b
0011	0 1 0 0 1 1 1 1	4f
0100	0 1 1 0 0 1 1 0	66
0101	0 1 1 0 1 1 0 1	6d
0110	0 1 1 1 1 1 0 1	7d
0111	0 0 0 0 0 1 1 1	07
1000	0 1 1 1 1 1 1 1	7f
1001	0 1 1 0 1 1 1 1	6f



```
ORG 0000H          ; Start of program memory
MOV P2, #00H       ; Clear Port 2 (connected to 7-segment)
MOV DPTR, #TAB     ; Load address of lookup table

MOV R0, #0         ; Counter for digits 0–9

BACK:
   MOV A, R0
   MOVC A, @A+DPTR ; Get segment pattern from table
   MOV P2, A       ; Output to 7-segment display

   ACALL DELAY     ; Small delay

   INC R0
   CJNE R0, #10, BACK ; Repeat for digits 0–9

   SJMP $          ; Stop program (loop here)
 ; -------------------------
 ; Delay Subroutine
 DELAY:
    MOV R1, #200
 L1: MOV R2, #255
 L2: DJNZ R2, L2
    DJNZ R1, L1
    RET
 ; -------------------------

 ; Lookup table for digits 0–9
 TAB:
 DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 6FH
 END
```

8051 Port Configuration

1) It is suggested to configure the I/O Ports of 8051 before its usage

2) To configure **Port1 as input port**

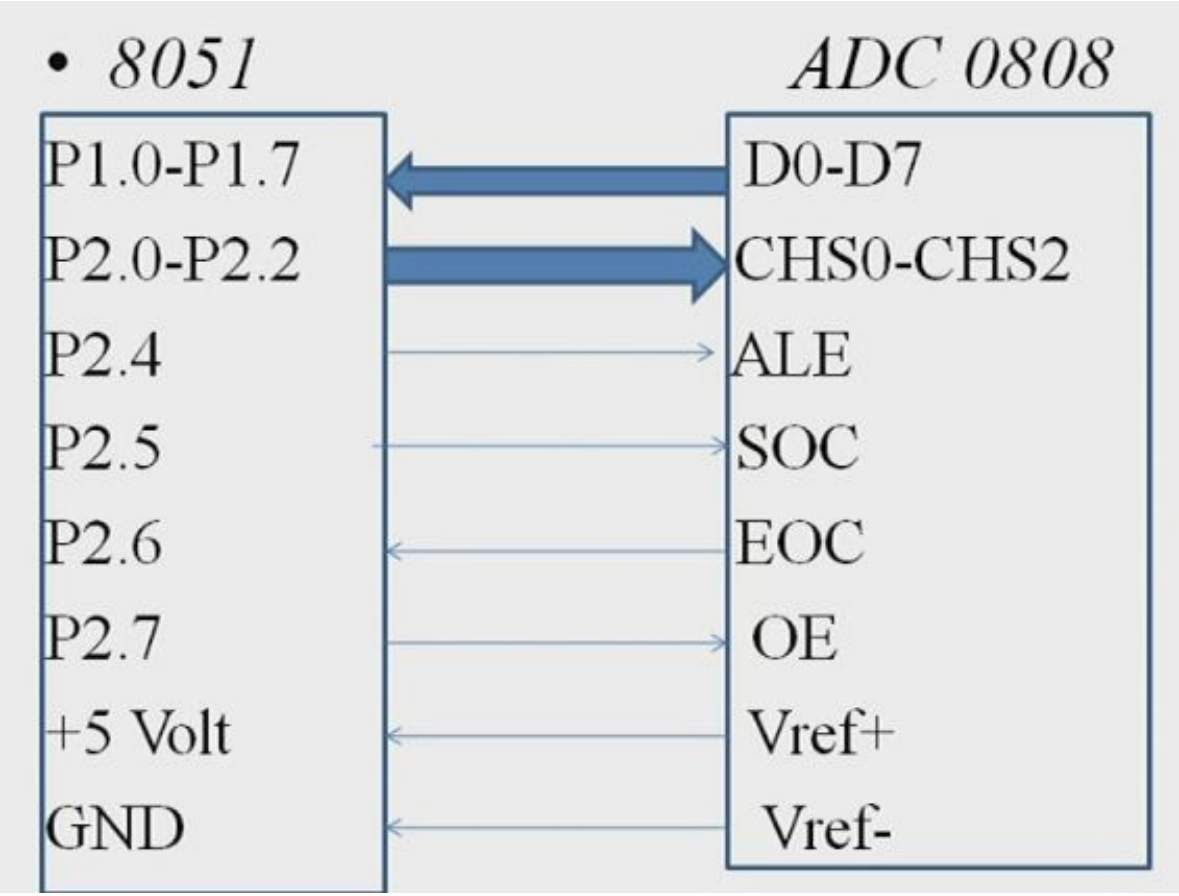
MOV P1, #0FFH

3) To configure **Port1 as output port**

MOV P1, #00H

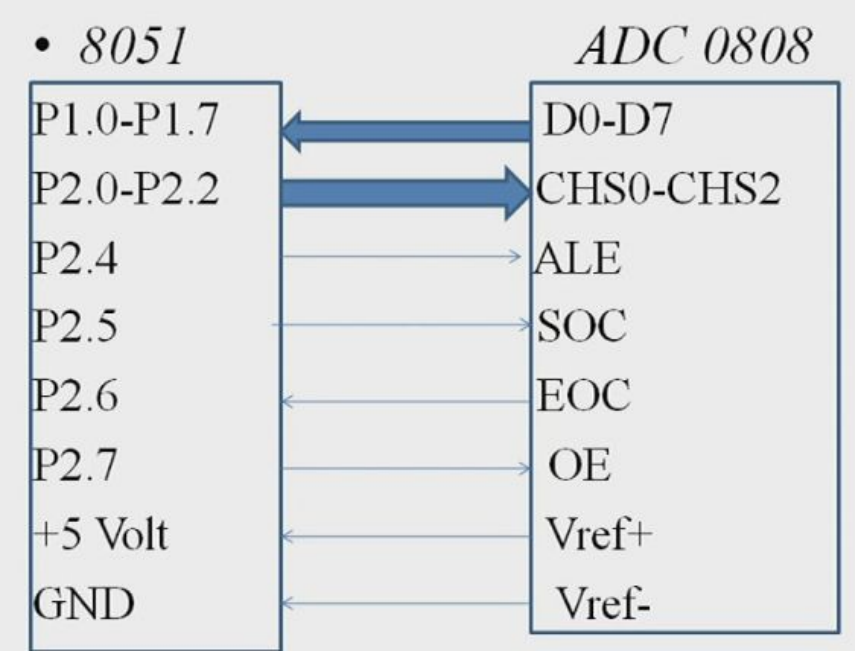
4) It need to be done at the beginning of the ASM program

ADC interface



ADC interface procedure

- **Step 1:** Set the port you connected to the output lines of the ADC as an input port [Port1].
- **Step 2:** Make the Port connected to EOC pin high. The reason for doing this is that the ADC sends a high to low signal when the conversion of data is complete
- **Step 3:** ADC channel section will be done by sending 3 bits through P2.0-P2.2
- **Step 4:** Select the data lines according to the input port you want to select. To do this, select the data lines and send a High to Low pulse at the ALE pin to select the address.
- **Step 5:** Now that we have selected the analog input pin, we can tell the ADC to start the conversion by sending a pulse to the SOC pin.
- **Step 6:** Wait for the High to low signal by polling the EOC pin.
- **Step 7:** Wait for the signal to get high again.
- **Step 8:** Extract the converted data by sending a High to low signal to the OE pin (Output Enable).



ADC Interface Program

- **Step 1:** Set the port you connected to the output lines of the ADC as an input port [Port1].

```
ORG 0000H; Starting address
MOV P1,#0FFH; Makes port 1 i/p port
```

- **Step 2:** Make the Port connected to EOC pin high. The reason for doing this is that the ADC sends a high to low signal when the conversion of data is complete, clear SOC, ALE, OE pins

```
SETB P2.6; Makes EOC pin high
CLR P2.4; Clears ALE pin
CLR P2.5; Clears Start SOC pin
CLR P2.7; Clears OE pin
```

- **Step 3:** ADC channel section will be done by sending 3 bits through P2.2-P2.0 **[010 – CH2]**

```
BACK:    CLR P2.2;
         SETB P2.1;
         CLR P2.0;
```

- **Step 4:** Select the data lines according to the input port you want to select. To do this, select the data lines and send a High to Low pulse at the ALE pin to select the address.

```
SETB P2.4; Sets ALE high

CLR P2.4; makes ALE low
```

- **Step 5:** Now, we have selected the analog input pin, we can tell the ADC to start the conversion by sending a pulse to the SOC pin.

```
SETB P2.5; to start conversion
CLR P2.5; makes Start pin low HERE
```

- **Step 6:** Wait for the High to low signal by polling the EOC pin.

```
HERE: JNZ P2.6,HERE;
```

- **Step 7:** Wait for the signal to get high again.

```
HERE1: JZ P2.6,HERE1;
```

Step 8: Extract the converted data by sending a High to low signal to the OE pin

```
SETB P2.7; enables OE pin
MOV A,P1;
CLR P2.7; clears OE
SJMP BACK;
```

Another method for step 6 and 7

- Use the concept of Delay
- Lets assume that, we need 100 microseconds delay [for conversion as per ADC IC requirement]
- We know that, one register is good enough
- Consider the operating frequency of 8051 as 12MHz

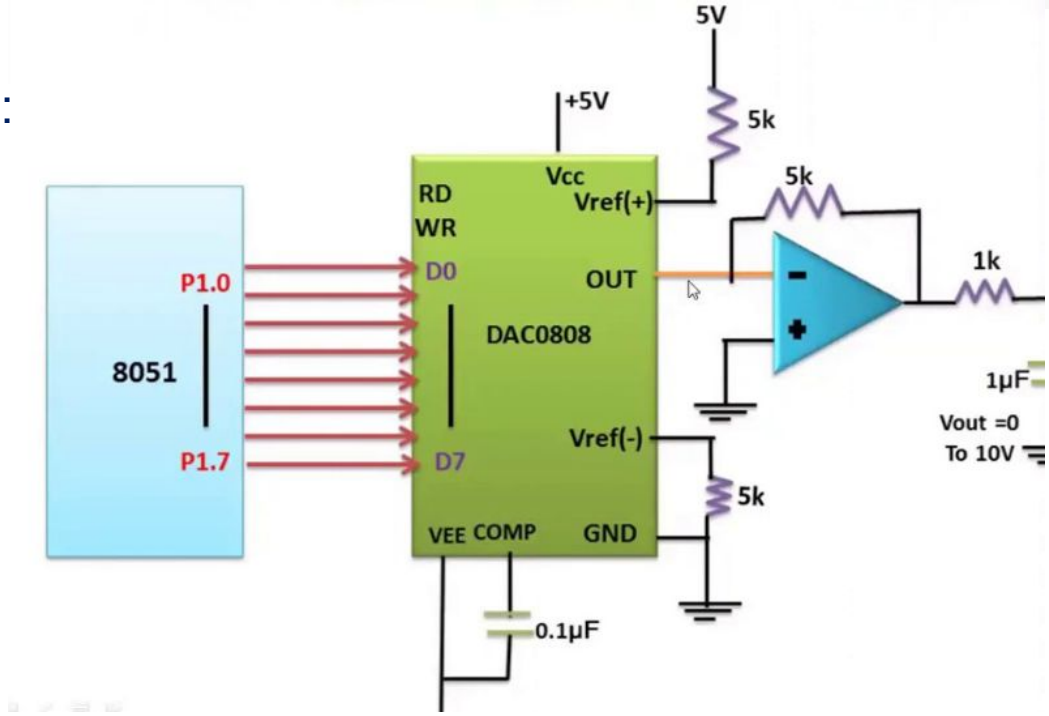
```
DELAY:      MOV R3,#______
L1:         DJNZ R3,L1
            RET

            CALL DELAY
      L2:   MOV C, P2.6
            JNC  L2
```

DAC interface

Procedure for square wave form:

1. Clear P1
2. Call delay
3. Set P1
4. Call delay
5. Repeat from step1



Program for square wave form:

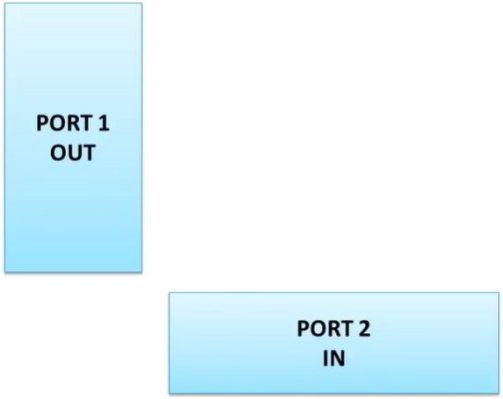
```
            ORG 0000H
REPEAT:     MOV P1, #00H
            LCALL D1
            MOV P1, #0FFH
            LCALL D1
            SJMP REPEAT
```

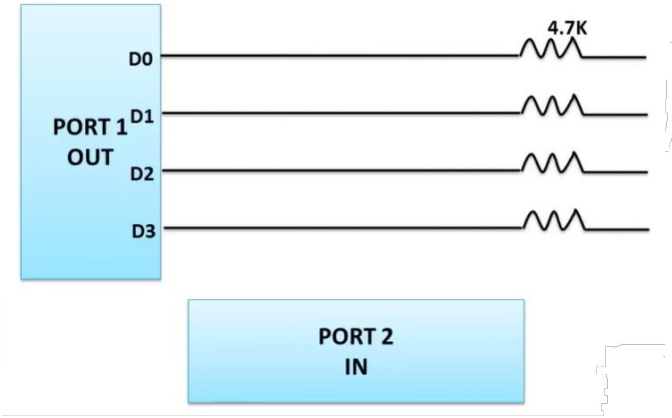
D1: write delay program here as per pulse width

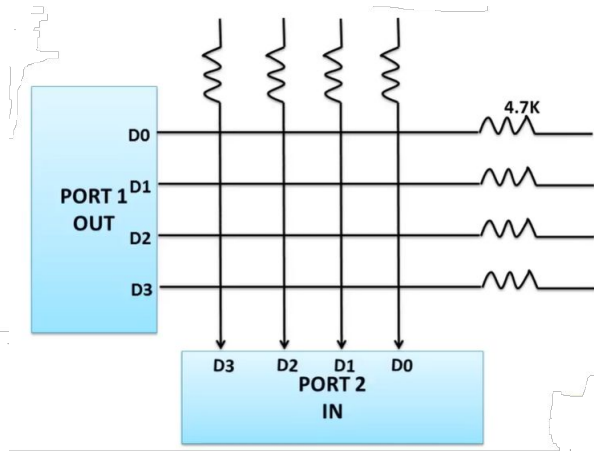
View this for generation of triangular wave form using DAC

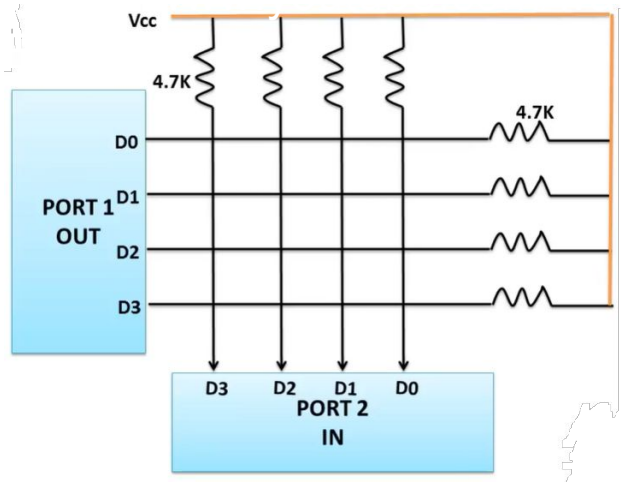
https://www.youtube.com/watch?v=xV3lUy0sUHI

KEYBOARD interface



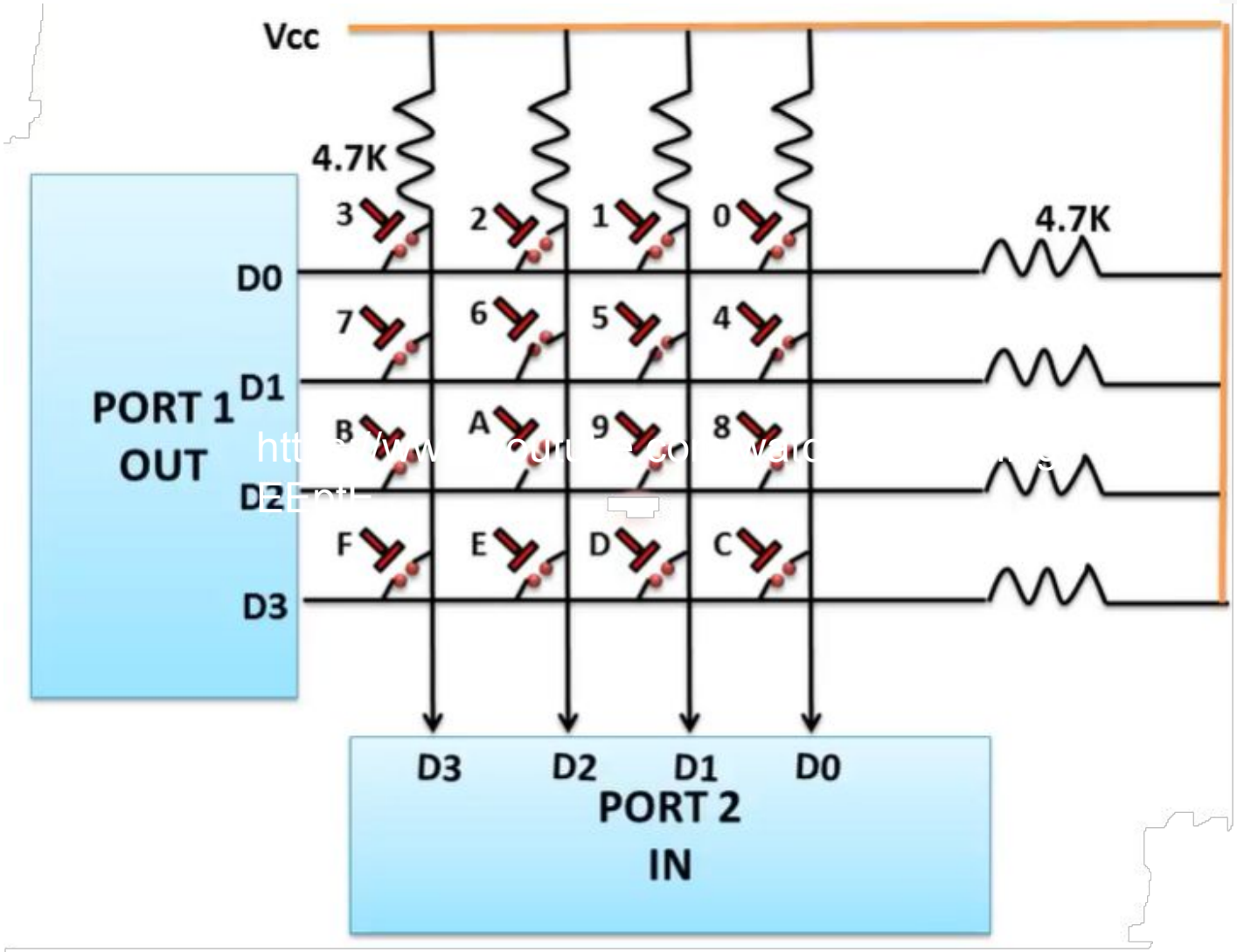






https://www.youtube.com/watch?v=0xHdq0QS1aQ

KEYBOARD interface



VIT BHOPAL UNIVERSITY

16 keys arranged as a 4X4 matrix

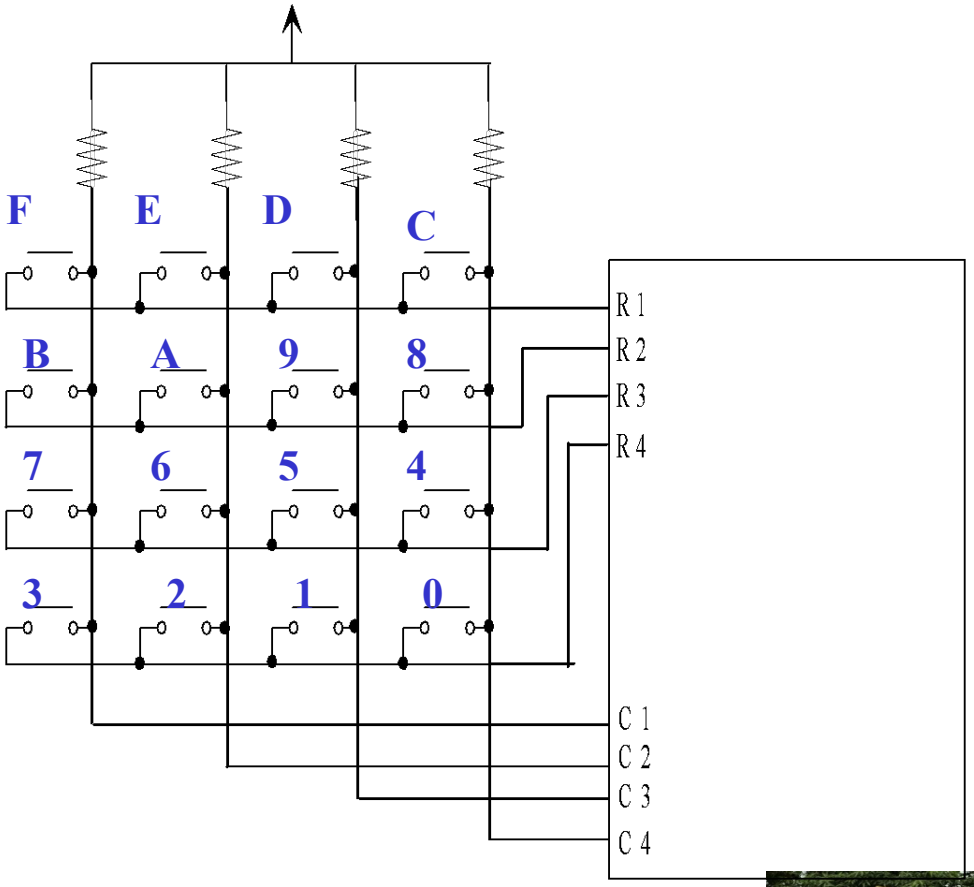
Place a 0 on R0 port

Read C port

If there is a 0 bit
then the button
at the column/row
intersection has
been pressed.

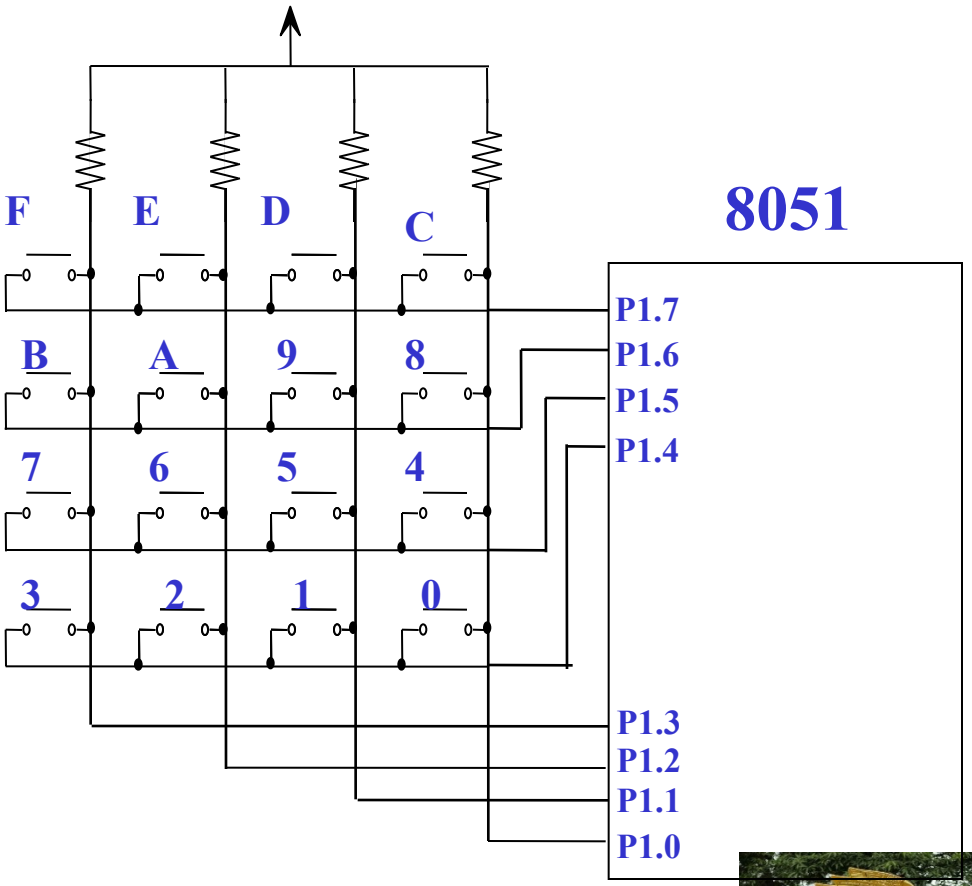
Otherwise, try next row

Repeat constantly



Interfacing a Keypad

```
scan:   mov P1,#EFH
        jnb P1.0,db_0
scan1: jnb P1.1,db_1
scan2: jnb P1.2,db_2
scan3: jnb P1.3,db_3
scan4: mov P1,#DFH
        jnb P1.0,db_4
…..
…..
…..
```



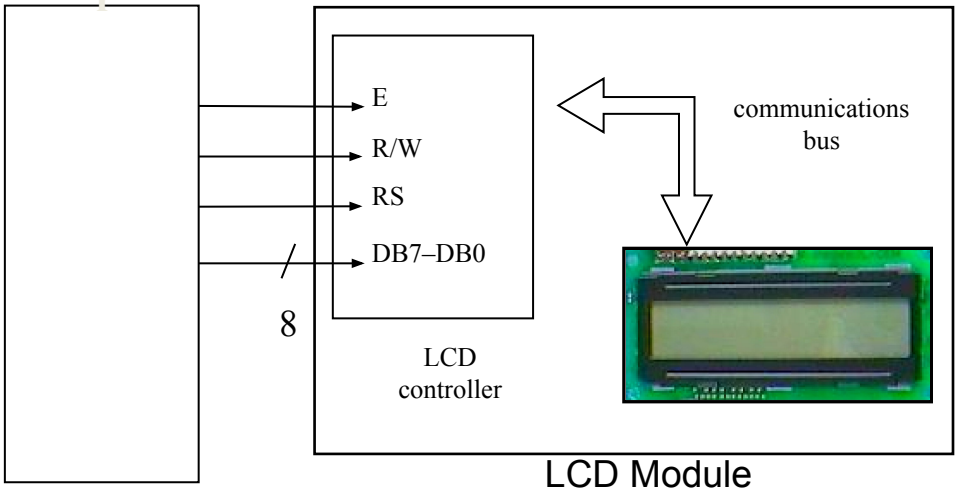
Interfacing a Keypad

```
db_0:      lcall  wt_10ms
       jb  P1.0, scan1
       mov A, #0
       ljmp   get_code
db_1:      lcall  wt_10ms
       jb  P1.1, scan2
       mov A, #1
       ljmp   get_code
…..
…
…..
get_code:movDPTR, #key_tab
       movc   A, @A+DPTR
       ljmp   scan
key_tab: db '0123456789ABCDEF'
       END
```

LCD Interfacing

- Liquid Crystal Displays (LCDs)
- cheap and easy way to display text
- Various configurations (1 line by 20 X char up to 8 lines X 80)
- Integrated controller
- The display has two register
  - command register
  - data register
- By RS you can select register
- Data lines (DB7-DB0) used to transfer data and commands

Alphanumeric LCD Interfacing



- Pinout
  - 8 data pins D7:D0
  - RS: Data or Command Register Select
  - R/W: Read or Write
  - E: Enable (Latch data)

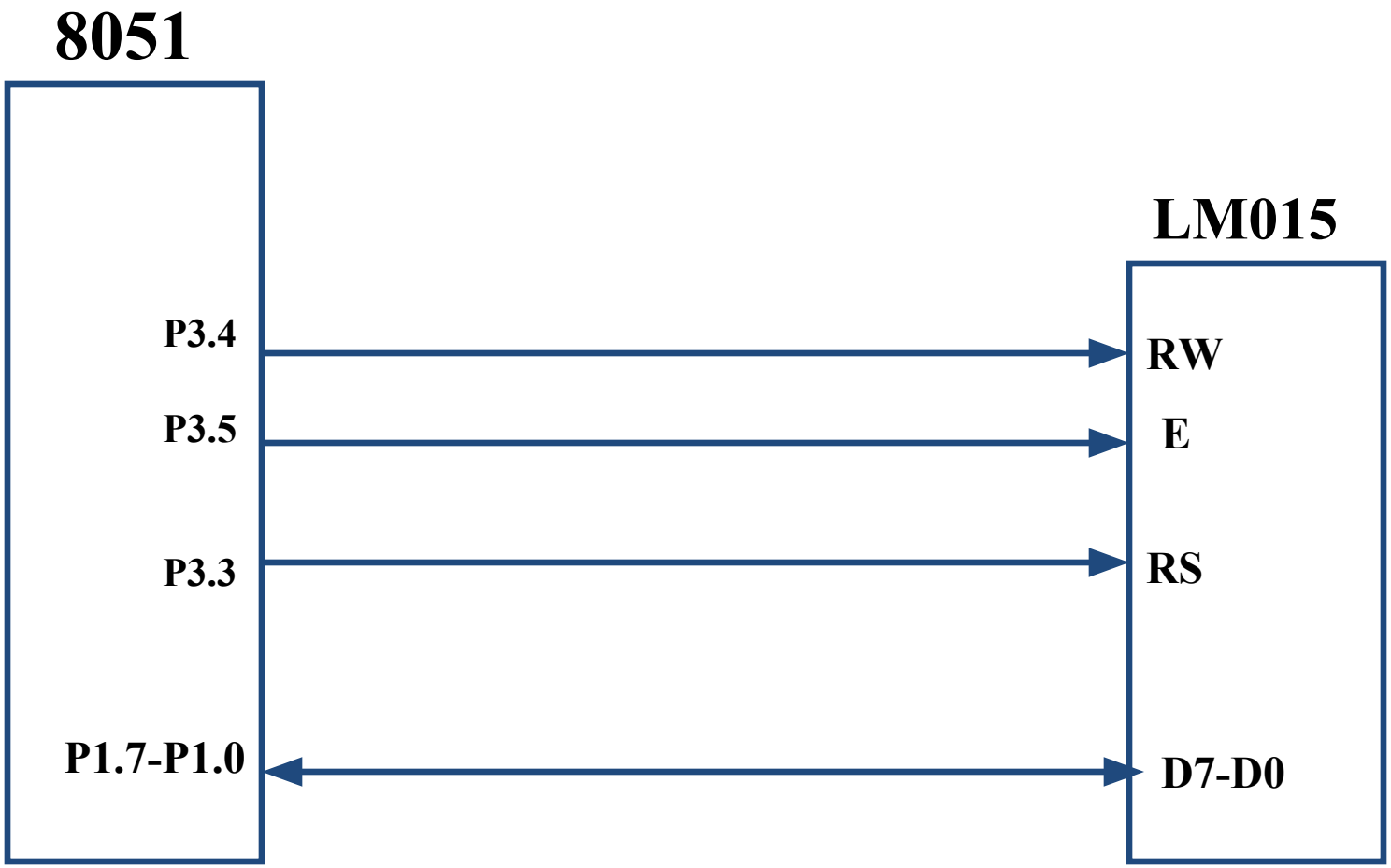
- RS – Register Select
  - RS = 0 → Command Register
  - RS = 1 → Data Register
- R/W = 0 → Write , R/W = 1 → Read
- E – Enable
  - Used to latch the data present on the data pins.

- D0 – D7
  - Bi-directional data/command pins.
  - Alphanumeric characters are sent in ASCII format.

Table 4-10: List of Instructions (Courtesy of Optrex Corporation)

Instruction	Code										Description	Execution Time (max)
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0		
Clear Display	0	0	0	0	0	0	0	0	0	1	Clears entire display and sets DD RAM address 0 in address counter.	1.64 ms
Return Home	0	0	0	0	0	0	0	0	1	--	Sets DD RAM address 0 as address counter. Also returns display being shifted to original position. DD RAM contents remain unchanged.	1.64 ms
Entry Mode Set	0	0	0	0	0	0	0	1	I/D	S	Sets cursor move direction and specifies shift of display. These operations are performed during data write and read.	40 µs
Display ON/OFF Control	0	0	0	0	0	0	1	D	C	B	Sets ON/OFF of entire display (D), cursor ON/OFF (C), and blink of cursor position character (b).	40 µs
Cursor or Display Shift	0	0	0	0	0	1	S/C	R/L	--	--	Moves cursor and shifts display without changing DD RAM contents.	40 µs
Function Set	0	0	0	0	1	DL	N	F	--	--	Sets interface data length (DL), number of display lines (L) and character font (F).	40 µs
Set CG RAM Address	0	0	0	1	AGC						Sets CG RAM address. CG RAM data is sent and received after this setting.	40 µs
Set DD RAM Address	0	0	1	ADD							Sets DD RAM address. DD RAM data is sent and received after this setting.	40 µs
Read Busy Flag & Address	0	1	BF	AC							Reads Busy flag (BF) indicating internal operation is being performed and reads address counter contents.	40 µs
Write Data to CG or DD RAM	1	0	Write Data								Writes data into DD RAM or CG RAM.	40 µs
Read Data from CG or DD RAM	1	1	Read Data								Reads data from DD RAM or CG RAM.	40 µs

Interfacing LCD with 8051



Interfacing LCD with 8051

```
mov A, command
call cmd
delay
mov A, another_cmd
call cmd
delay
mov A, #'A'
call data
delay
mov A, #'B'
call data
delay
….
Command and Data Write Routines
data:mov P1, A   ;A is ascii data
     setb P3.3   ;RS=1 data
     clr P3.4    ;RW=0 for write
     setb P3.5   ;H->L pulse on E
     clr P3.5
     ret
cmd:mov P1,A     ;A has the cmd word
     clr P3.3    ;RS=0 for cmd
     clr P3.4    ;RW=0 for write
     setb P3.5   ;H->L pulse on E
     clr P3.5
     ret
```